

Haute disponibilité

ELB + Auto Scaling : un service qui survit à la panne

CL-AWS1 — Jour 6 — Aujourd'hui, on tue une instance en direct

Objectifs de la journée

À la fin de la journée, vous saurez :

- Expliquer pourquoi **une seule instance ne suffit jamais** (panne, patch, pic).
- Décrire un **ALB** : listeners, target groups, health checks — et le placer dans la chaîne des security groups.
- Construire un **Auto Scaling Group** : launch template, min/désiré/max, politique **target tracking**.
- Diagnostiquer un target group **unhealthy** avec méthode.
- Dire pourquoi l'application doit être **stateless** — et où déplacer sessions et fichiers.
- Dessiner l'**architecture 3-tiers complète** : celle du TP2, votre TP noté.

Le contrat du jour : en fin de journée, vous tuez une instance **et le service ne tombe pas**.

Plan de la journée

1. **Pourquoi une seule instance ne suffit jamais** — les trois morts d'un serveur.
2. **ELB** — types, ALB en détail : listeners, target groups, health checks.
3. **Auto Scaling Groups** — launch template, tailles, politiques, cycle de vie.
4. **L'application stateless** — où vont les sessions, où vont les fichiers.
5. **Démo** — ALB + ASG, test de panne chronométré.
6. **L'architecture 3-tiers** — le plan du TP2, pièce par pièce.

Fin de journée : **mini-TP en autonomie** (non noté) — votre répétition générale.

1. Pourquoi une seule instance ne suffit jamais

Les trois morts d'un serveur seul

Votre StockLine du J5 tourne sur UNE instance. Elle mourra trois fois :

1. **La panne** — disque, hôte physique, AZ entière. Pas « si », mais « quand ». Une instance seule = service par terre + réveil à 3 h du matin (souvenez-vous du TP1).
2. **La maintenance** — patcher le noyau, changer de type d'instance, redéployer l'application : sur une instance seule, chaque opération = interruption planifiée... ou repoussée éternellement.
3. **Le succès** — le pic de trafic (promo, presse, lundi matin). Une instance a un plafond ; au-delà : lenteurs, timeouts, erreurs.

Trois problèmes différents, une seule cause : **le singulier**. La solution tient en un mot : **plusieurs** — et quelque chose pour les orchestrer.

La disponibilité se chiffre

Le vocabulaire contractuel (les « neufs ») :

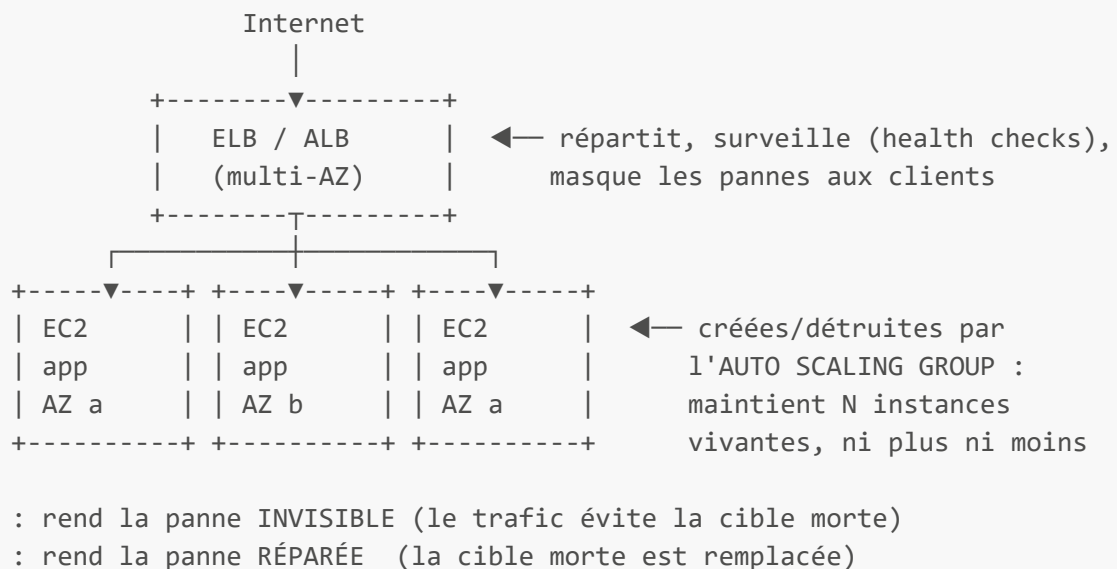
Disponibilité	Indisponibilité max/an	Réaliste avec...
99 %	~3,7 jours	une instance et de la chance
99,9 %	~8,8 heures	2 instances, 2 AZ, bascules auto
99,99 %	~53 minutes	multi-AZ partout + exploitation rodée

Deux définitions à séparer (elles se confondent souvent) :

- **Haute disponibilité** : le service survit aux **pannes** (redondance).
- **Élasticité** : la capacité suit la **charge** (scaling).

L'ALB sert les deux ; l'ASG implémente les deux. D'où la journée entière sur ce couple.

La réponse AWS : deux briques



2. ELB — Elastic Load Balancing

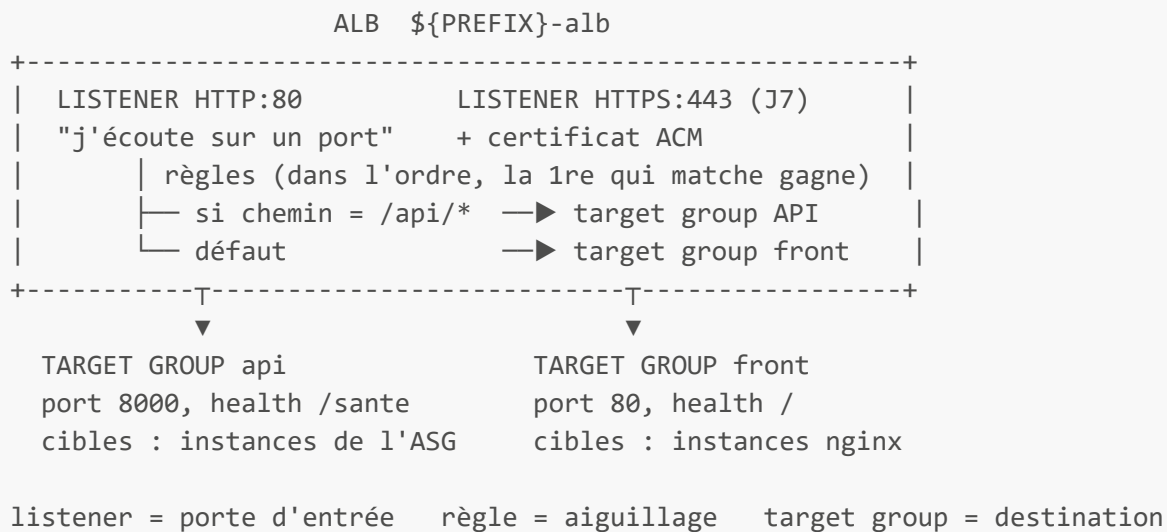
Les types de load balancers

ELB est une famille — on choisit selon la couche réseau (rappel CL-RÉSEAU) :

- **ALB — Application Load Balancer** (couche 7, HTTP/HTTPS) : comprend les URL, les en-têtes, les codes de statut. Routage par chemin (`/api/*`) ou par nom d'hôte. **Notre outil pour StockLine et le TP2.**
- **NLB — Network Load Balancer** (couche 4, TCP/UDP) : ne lit pas le HTTP, mais des millions de connexions, latence minimale, **IP fixe possible** par AZ. Pour les protocoles non-HTTP, les débits extrêmes.
- **GWLB — Gateway Load Balancer** : cas spécial (chaîner des appliances de sécurité) — citez-le, n'en dites pas plus à ce stade.

Règle simple : application web → **ALB** ; TCP brut, IP fixe exigée, performance extrême → **NLB**.

Anatomie d'un ALB



Le target group et ses cibles

Le **target group** = la liste des destinations + la règle pour juger leur santé.

- **Type de cible** : **instance** (nos EC2), **ip** (conteneurs, plus tard), **lambda** (CL-AWS2).
- Une cible = une instance + **un port** (notre API écoute sur 8000).
- L'ASG peut **inscrire et désinscrire les cibles automatiquement** (option `--target-group-arns` à la création de l'ASG) — personne ne gère la liste à la main.

```

aws elbv2 create-target-group \
  --name ${PREFIX}-tg-app --protocol HTTP --port 8000 \
  --vpc-id $VPC_ID --target-type instance \
  --health-check-path /sante

```

`/sante` : le health check de StockLine existe **depuis CL-PYTHON**. Ce n'était pas du zèle — c'était pour aujourd'hui.

Health checks : le juge de paix

Toutes les N secondes, l'ALB appelle chaque cible : `GET /sante` → il attend un **200**.

Paramètre	Défaut	Notre réglage	Rôle
<code>interval</code>	30 s	15 s	fréquence de la sonde
<code>unhealthy-threshold</code>	2	2	échecs consécutifs → sortie
<code>healthy-threshold</code>	5	2	succès consécutifs → retour
<code>timeout</code>	5 s	5 s	délai max de réponse

- Cible `unhealthy` → **sortie de la rotation** (plus aucun trafic) → détection en ~30 s avec nos réglages.
- États : `initial` (en cours d'inscription) → `healthy` / `unhealthy` → `draining` (en cours de retrait, on laisse finir les requêtes en vol).
- Piège vu à l'exercice 6-2 : une app qui répond 200 en étant malade est jugée saine. **Le health check doit dire la vérité** (503 si la base est morte).

L'ALB dans le réseau : multi-AZ et SG chaînés

Deux règles de placement, non négociables :

- L'ALB exige **au moins 2 sous-réseaux dans 2 AZ différentes** (il est lui-même redondant — pas de point de défaillance unique chez le répartiteur).
- Chaîne de confiance par les security groups :

```

Internet —80/443→ [ sg-alb ] —port app→ [ sg-app ] —5432→ [ sg-db ]
0.0.0.0/0 autorisé source = source =
ICI et SEULEMENT ici sg-alb sg-app
  
```

Personne ne parle aux instances sans passer par l'ALB ; personne ne parle à la base sans passer par les instances. Trois SG, zéro IP en dur : le patron de sécurité du TP2.

💰 L'ALB n'est PAS dans le free-tier

💰 Encadré coût — ALB (eu-west-3)

- **~0,027 \$/heure** dès la création, même à trafic NUL ;
- ◦ **~0,008 \$/LCU-heure** selon l'usage (connexions, débit, règles) ;
- **≈ 0,70 /jour * *, ≈ * * 20/mois** pour un ALB qui dort ;
- un ALB oublié un week-end ≈ 2 \$. Par apprenant. Par ALB.

Conséquences pratiques pour le bloc :

- l'ALB se crée **le matin**, se détruit **le soir** (`teardown-alb-asg.sh`) ;
- le script refuse de créer un 2^e ALB si `alb-ids.env` existe : c'est un garde-fou, pas une brimade ;

- au J8, votre script d'inventaire boto3 signalera tout ALB survivant.

3. Auto Scaling Groups

Le launch template : le moule à instances

L'ASG ne clone pas des machines : il **ré-exécute une recette**. Cette recette, c'est le **launch template** :

- AMI (notre Ubuntu 24.04 via paramètre SSM), type d'instance (t3.micro) ;
- security group, key pair, **profil IAM** (le rôle S3 du J4) ;
- **user data** : le script qui installe StockLine au boot (J2 + J5).

Conséquences à intégrer :

- toute instance de l'ASG naît **identique et automatiquement configurée** — si l'installation exige un geste manuel, l'autoscaling est cassé par construction ;
- le template est **versionné** : on ne modifie pas, on publie une version et l'ASG prend **\$Latest** — les nouvelles instances l'utilisent, les anciennes non (le « refresh » viendra avec l'IaC) ;
- l'ancêtre *launch configuration* existe dans les vieilles docs : obsolète, ne plus utiliser.

min / désiré / max : les trois curseurs

min = 2	désiré = 2	max = 4
"jamais moins"	"l'objectif actuel"	"jamais plus"
= haute dispo	= ajusté par les	= plafond de
(2 AZ couvertes)	politiques de scaling	FACTURE et garde-fou

L'ASG travaille en boucle infinie :

```

état réel (instances saines) == désiré ?
trop peu → LANCE une instance (launch template)
trop     → TERMINE une instance
pile     → dort et re-vérifie
  
```

`--vpc-zone-identifiant subnet-a,subnet-b` : l'ASG répartit et RÉÉQUILIBRE les instances entre les AZ automatiquement`

Qui décide qu'une instance est morte ?

L'ASG a deux sources de vérité possibles — le choix compte :

- `--health-check-type EC2` (défaut) : l'avis de l'**hyperviseur**. Une VM qui boote = vivante... même si l'application a planté dedans. **Les zombies passent.**

- `--health-check-type ELB` : l'avis du **health check applicatif** de l'ALB. Une app qui ne répond plus 200 → instance déclarée morte → **remplacée**. Notre choix, toujours, derrière un ALB.

Le paramètre qui va avec : `--health-check-grace-period 180` — le délai de grâce pendant lequel l'ASG **ignore** les avis de santé, le temps que cloud-init finisse.

Trop court = l'ASG tue les instances **pendant leur démarrage**, qui rebootent, qui sont tuées... le cimetière perpétuel (vous le verrez à l'exercice 6-2 — et peut-être en vrai au TP2).

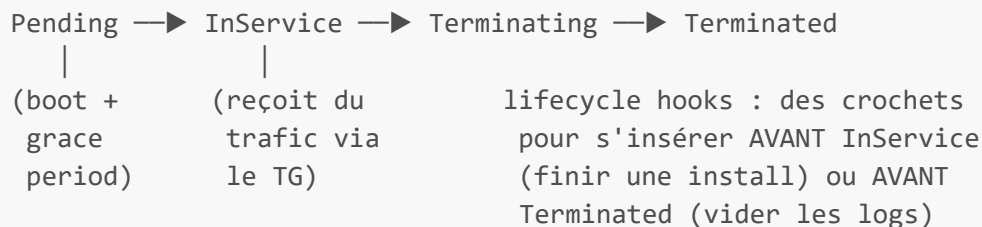
Les politiques de scaling

Trois façons de piloter le « désiré » :

- **Target tracking** ← notre choix : « maintiens la CPU moyenne à 50 % ». AWS crée les alarmes CloudWatch et ajuste tout seul, dans les deux sens. Le thermostat.
- **Step scaling** : « si CPU > 70 % ajoute 1, si > 90 % ajoute 3 ». Plus fin, plus manuel — quand le thermostat ne suffit plus.
- **Scheduled actions** : « vendredi 17 h 50, passe à 8 instances ». Pour la charge **prévisible** (promo annoncée) : on n'attend pas que la CPU monte, on anticipe.

Pourquoi une cible à 50 % et pas 90 % ? Il faut **de la marge pour tenir pendant les 3-5 min de démarrage** des renforts. À 90 %, quand l'alarme sonne, il est déjà trop tard. (Exercice 6-1 : vous allez le calculer.)

Le cycle de vie d'une instance d'ASG



Ce qu'il faut retenir au niveau du bloc :

- une instance d'ASG est **jetable par définition** : elle peut passer à **Terminating** à tout moment (scale-in, unhealthy, rééquilibrage d'AZ) ;
- les **lifecycle hooks** existent pour les cas fins (drainage, sauvegarde de logs) — sachez que c'est là, on ne les câble pas aujourd'hui ;
- corollaire immédiat : **rien d'important ne doit vivre sur l'instance**. Transition parfaite vers la section 4...



Exercice 6-1 — Dimensionner un ASG qui tient la route

Fichier : `exercices/06-cl-aws1/exercice-6-1-dimensionner-un-asg.md` — **30 min, en binômes**

Avec de vraies mesures (300 req/s par instance, pointes, promo ×2, boot 4 min) :

- calculer la capacité **utile** (à 60 % de CPU, pas 100 %) ;
- appliquer la règle « perte d'une AZ sans dégradation » (la formule $\times 3/2$) ;
- produire le triplet **min/désiré/max** argumenté + la cible de target tracking ;
- auditer 3 configurations vues « chez d'autres clients » — dont un cimetière perpétuel.

Sur papier, calculatrice permise. Les calculs posés comptent autant que les résultats : au TP2, ce dimensionnement argumenté est **dans la grille**.

4. L'application stateless

Où vont les sessions ? Où vont les fichiers ?

Le problème : l'état local meurt avec l'instance

Deux symptômes classiques dès qu'on met une application « ordinaire » derrière un ALB :

- **Déconnexions aléatoires** : la session utilisateur vit **en mémoire** de l'instance A ; la requête suivante atterrit sur l'instance B, qui ne la connaît pas. Une chance sur deux par clic.
- **Fichiers fantômes** : l'upload est écrit **sur le disque local** de A ; le téléchargement est servi par B. « Une fois sur deux, mon fichier n'existe pas. »

Et la version fatale : l'ASG **remplace** une instance (c'est son travail !) → tout ce qu'elle gardait localement disparaît. Sessions, uploads, caches, compteurs.

Règle à graver : **une instance d'ASG doit pouvoir mourir à tout instant sans rien emporter**. Si ce n'est pas vrai, l'état est au mauvais endroit.

Où va chaque état

```

+----- instance EC2 (JETABLE) -----+
| code de l'app, venv, config injectée par user data |
| RIEN d'autre. Aucune donnée. Aucun fichier utilisateur. |
+-----+
| sessions, | fichiers, | logs |
| données métier | images, exports |
| ▼ | ▼ | ▼ |
+-----+ +-----+ +-----+
| RDS (J5) | | S3 (J4) | | CloudWatch |
| l'état | | les objets | | Logs (J8) |
| structuré | | | | |
+-----+ +-----+ +-----+

(sessions à haute fréquence : cache externe ElastiCache – CL-AWS2)

```

Sticky sessions : le pansement, pas l'architecture

L'ALB propose l'**affinité de session** (« sticky sessions », cookie **AWSALB**) : chaque client est collé à « son » instance.

-  Ça masque le symptôme des déconnexions... tant que l'instance vit.
-  Quand l'ASG remplace l'instance (sa vocation !), **tous ses clients perdent leur session d'un coup**.
-  La charge se répartit mal : les clients collés ne migrent pas vers les nouvelles instances.
-  Et les fichiers locaux restent perdus : le cookie ne déplace pas les données.

Verdict : la sticky session dépanne une application legacy qu'on ne peut pas modifier. Pour StockLine — qui est déjà stateless : état dans PostgreSQL, front statique dans S3 — on n'en a **pas besoin**. C'était un choix de conception, dès CL-PYTHON.



Exercice 6-2 — Cibles unhealthy : le diagnostic derrière l'ALB

Fichier : `exercices/06-cl-aws1/exercice-6-2-diagnostic-target-group.md` — **40 min, seul(e)**

Vous êtes d'astreinte, le site renvoie 503 :

- construire la **checklist ordonnée** (SG → écoute → chemin → grace period → cloud-init) ;
- diagnostiquer trois relevés réels (**Target.Timeout**, **ResponseCodeMismatch**, remplacement en boucle) et écrire les **commandes CLI de correction** ;
- partie 2 : les symptômes du **stateful** (déconnexions, fichiers fantômes), le procès de la sticky session, la correction d'architecture.

C'est l'exercice le plus rentable de la semaine : ces pannes-là, vous les rencontrerez **au TP2** — autant les avoir déjà résolues sur papier.

5. Démo — Tuer une instance sans faire tomber le service



Démo 6-1 — ALB + ASG, le test de panne

Fichier : `demos/06-cl-aws1/demo-6-1-alb-asg.md` — **60 min**

Le programme, tout en CLI (`create-alb-asg.sh`, déroulé section par section) :

1. les SG chaînés, le rôle IAM, le **launch template** (user data du J2 : la page qui affiche l'ID d'instance) ;
2. target group + ALB + listener, et les cibles qui passent **initial** → **healthy** ;
3. la répartition **à l'œil nu** : `curl` en boucle, les IDs alternent ;
4. le **moment** : `terminate-instances` sur une cible — le service continue, l'ASG répare en ~4 min ;
5. option : la variante `APP_MODE=stockline` — deux API, une seule base RDS.

Pendant la démo, notez les DEUX durées : éviction (secondes) et réparation (minutes).

Ce que la démo a montré : deux mécanismes, deux horloges

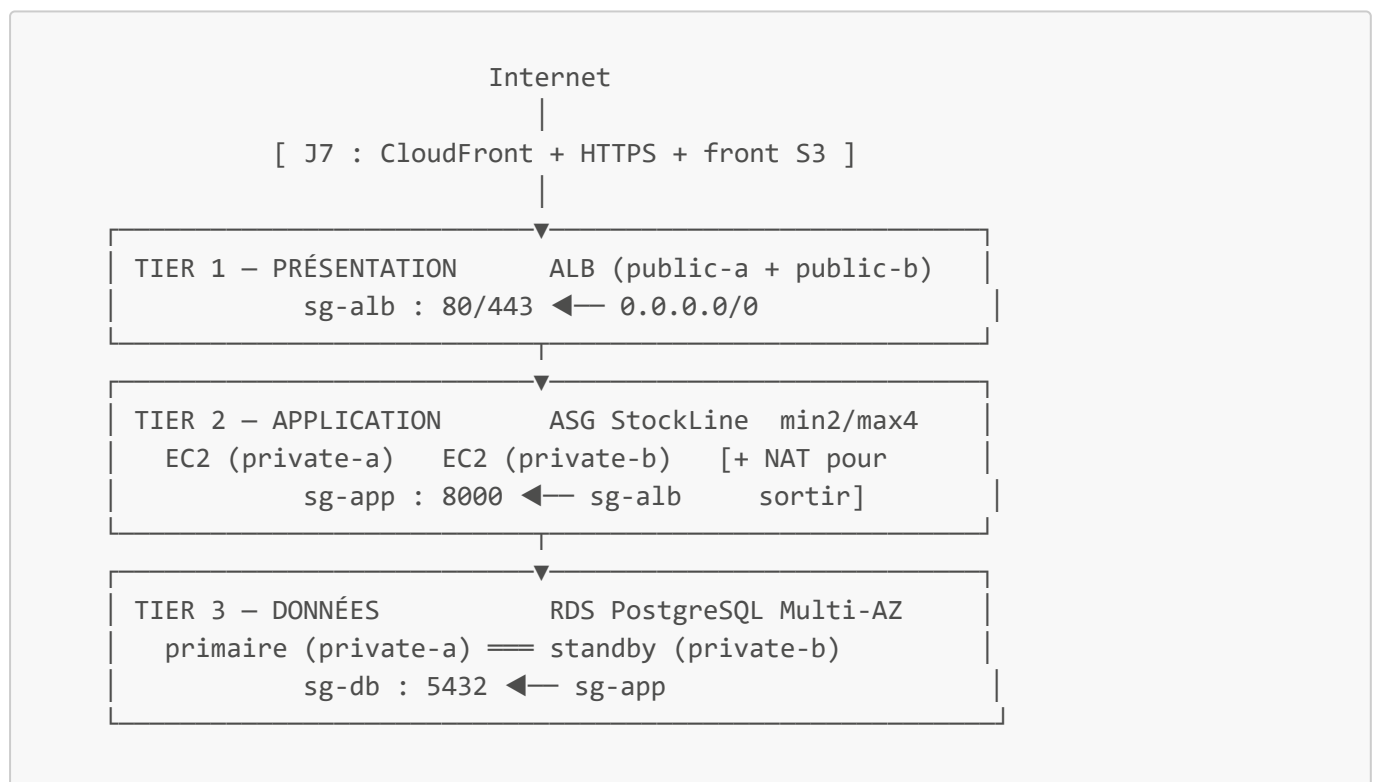
Événement	Acteur	Délai observé
La cible morte ne reçoit plus de trafic	ALB (health checks)	~15-30 s
Le service répond toujours (1 instance restante)	min = 2	continu
L'ASG constate 1 $\neq$ 2 et lance un remplacement	ASG	~1-2 min
La remplaçante sert du trafic (boot + healthy)	launch template + grace	~4-6 min

- L'ALB protège **l'utilisateur** (échelle : secondes).
- L'ASG répare **l'infrastructure** (échelle : minutes).
- Pendant la réparation, le service tourne **dégradé mais debout** — c'est exactement ce que **min=2** achète. Avec **min=1**, ces 4-6 minutes seraient une **panne totale**.

6. L'architecture 3-tiers complète

Le plan de votre TP2

Le schéma cible — à savoir dessiner les yeux fermés



Pourquoi trois tiers — et ce que chacun exige

Tier	Rôle	Ce qui le rend robuste
Présentation	recevoir le monde	ALB multi-AZ (natif), HTTPS (J7)
Application	la logique, StockLine	ASG ≥ 2 instances / 2 AZ, stateless , user data
Données	l'état, et rien que lui	RDS Multi-AZ , sauvegardes, jamais exposé

Chaque tier **scale et tombe indépendamment** : on peut doubler l'application sans toucher la base, perdre une instance sans perdre une donnée.

Différences démo → TP2 : instances en sous-réseaux **privés** (d'où la NAT GW 💰), base **Multi-AZ**, et le front S3 + CloudFront du J7 par-dessus. Vous avez déjà construit chaque brique — le TP2, c'est l'assemblage documenté.



Mini-TP — Un service qui survit à la panne (non noté)

Fichier : `tp/06-cl-aws1/tp-mini-ha.md` — **1 h 45, en autonomie**

Le défi du chef d'équipe : « reconstruis la démo tout seul, tue une instance, chronomètre la réparation, et laisse le compte VIDE ».

1. VPC (script J3) → ALB + ASG (script du jour, **lu et résumé** section par section) ;
2. preuve de la répartition (`uniq -c` sur 20 requêtes) ;
3. test de panne **chronométré** : T1 éviction, T2 réaction ASG, T3 nouvel ID ;
4. teardown scripté + **preuve par trois commandes vides**.

Livrable : un journal de bord d'une page — le brouillon de votre compte rendu du TP2. 💰 ALB facturé : le teardown fait partie du TP.

Récap — les points clés du jour

- Une instance seule meurt trois fois : **panne, maintenance, succès**. La haute dispo commence à **n ≥ 2 , sur 2 AZ**.
- **ALB** : listener → règles → target group ; le **health check** décide qui reçoit du trafic ; SG chaînés Internet → ALB → app → base.
- **ASG** : launch template (le moule) + min/désiré/max + `health-check-type ELB` + grace period ; **target tracking** = thermostat, **scheduled** = anticipation.
- L'ALB rend la panne **invisible** (secondes) ; l'ASG la rend **réparée** (minutes) ; `min=2` paie l'écart entre les deux.
- **Stateless obligatoire** : sessions → base, fichiers → S3, logs → CloudWatch. Sticky session = pansement.
- 💰 ALB ~0,70 \$/jour même vide : création le matin, **teardown le soir**.



Teardown du soir — non négociable

```
# 1. ALB + ASG + instances + SG + launch template :
./teardown-alb-asg.sh
```

```
# 2. Le VPC et sa NAT Gateway 💰 :  
./teardown-vpc.sh  
  
# 3. La preuve (trois sorties vides attendues) :  
aws elbv2 describe-load-balancers --region eu-west-3 \  
  --query 'LoadBalancers[].LoadBalancerName'  
aws ec2 describe-instances --region eu-west-3 \  
  --filters "Name=instance-state-name,Values=pending,running,stopping,stopped" \  
  --query 'Reservations[].Instances[].InstanceId'  
aws ec2 describe-nat-gateways --region eu-west-3 \  
  --filter "Name=state,Values=available" --query 'NatGateways[].NatGatewayId'
```

Si la base RDS du J5 tourne encore : décision explicite (garder pour J7 ? détruire ?) — jamais d'oubli par défaut.

Quiz du jour (1/5)

Question 1 — Citez les trois raisons pour lesquelles une application de production ne doit jamais reposer sur une seule instance EC2, même très puissante.

Question 2 — Quelle est la différence de rôle entre l'ALB et l'ASG face à une panne d'instance ? Donnez l'ordre de grandeur des délais de chacun.

Quiz du jour (2/5)

Question 3 — Dans un ALB, quel est le rôle respectif du listener, de la règle et du target group ? Illustrez avec le trajet d'une requête `GET /api/produits`.

Question 4 — Un target group affiche `unhealthy` avec `Reason: Target.Timeout`. Le SG des instances autorise le port 8000 depuis l'IP du poste de l'administrateur. Quel est le diagnostic et quelle est la correction ?

Quiz du jour (3/5)

Question 5 — À quoi sert le `health-check-grace-period` d'un ASG, et que se passe-t-il s'il est plus court que la durée réelle du cloud-init de l'application ?

Question 6 — Pourquoi une politique target tracking avec une cible CPU de 90 % est-elle une mauvaise idée pour une application dont les instances mettent 4 minutes à démarrer ?

Quiz du jour (4/5)

Question 7 — Votre trafic va doubler samedi à 14 h précises (opération commerciale annoncée). Quel mécanisme d'ASG utilisez-vous et pourquoi le target tracking seul ne suffit-il pas ?

Question 8 — Après le passage derrière un ALB, les utilisateurs sont déconnectés aléatoirement et leurs fichiers téléversés disparaissent une fois sur deux. Expliquez les deux symptômes et donnez la correction d'architecture pour chacun.

Quiz du jour (5/5)

Question 9 — Quelle est la différence entre `--health-check-type EC2` et `--health-check-type ELB` pour un ASG, et lequel choisit-on derrière un ALB ? Décrivez le scénario de panne que le mauvais choix laisse passer.

Question 10 — Dessinez (ou décrivez) l'architecture 3-tiers du TP2 : les trois tiers, le contenu de chacun, et la chaîne des trois security groups.

Réponses détaillées : guide formateur du bloc (jours 5-8).

À demain !

Jour 7 — Exposition au monde

Votre service est haute-dispo... mais son adresse est `abc-alb-1234.eu-west-3.elb.amazonaws.com`, en HTTP. Demain : **Route 53** (de vrais noms), **CloudFront** (le CDN), **ACM** (HTTPS partout) — et le front StockLine servi depuis un bucket S3 **privé**.

✂ Avant de partir : `teardown-alb-asg.sh` + `teardown-vpc.sh` exécutés, preuves vides.